Software Production Engineering

Mini Project Report

Scientific Calculator using DevOps CI/CD Pipeline

Academic Year
2025 – 2026

Submitted By

Rishu Agrawal
Roll No: MT2025104

Course
M.Tech CSE

Submitted To
(Professor B. Thangaraju)

International Institute of Information & Technology, Bangalore

Abstract

This project demonstrates the practical implementation of DevOps practices using a scientific calculator application as a case study. The primary objective of the project is to automate the software development lifecycle by integrating various modern DevOps tools and methodologies. DevOps focuses on improving collaboration between development and operations teams while enabling faster, more reliable, and continuous delivery of software. By automating tasks such as building, testing, packaging, and deployment, DevOps reduces manual intervention, minimizes errors, and improves the efficiency of the overall development process.

The application used in this project is a Java-based scientific calculator capable of performing basic arithmetic operations such as addition, subtraction, multiplication, and division, along with other mathematical computations. The application is developed using the Java programming language and is built using Apache Maven, which is a powerful build automation tool used for compiling source code, managing dependencies, running tests, and packaging the application into a deployable JAR file. Maven ensures a standardized project structure and simplifies the build process, making it easier to manage large-scale software projects.

For source code management, the project is integrated with GitHub, which serves as a centralized repository for storing and managing the application code. GitHub allows developers to maintain version control, track code changes, collaborate efficiently, and maintain a complete history of project updates. Whenever new code is pushed to the repository, it acts as a trigger for the automated pipeline.

To implement Continuous Integration and Continuous Deployment (CI/CD), Jenkins is used as the automation server. Jenkins continuously monitors the GitHub repository and automatically triggers the pipeline whenever changes are detected. The pipeline performs several automated tasks such as pulling the latest code from GitHub, building the project using Maven, running the required build steps, and preparing the application for deployment. This automation ensures that the software is continuously tested and built, reducing the chances of integration issues and improving the reliability of the software delivery process.

Once the application is successfully built, Docker is used to containerize the application. Containerization packages the application along with its dependencies into a lightweight and portable container, ensuring that the application runs consistently across different environments such as development, testing, and production. Docker eliminates environment-related issues by providing a standardized runtime environment.

After building the Docker image, the image is pushed to **Docker Hub**, which acts as a cloud-based container registry. Docker Hub stores the container images and allows them to be easily shared and deployed across different systems.

For automated deployment, Ansible is used as a configuration management and automation tool. Ansible simplifies the deployment process by executing predefined playbooks, which contain instructions for pulling the Docker image from Docker Hub and running the container on the target system. This ensures that the deployment process is fully automated and repeatable.

Overall, this project provides practical exposure to the complete DevOps workflow, including version control, continuous integration, containerization, container registry management, and automated deployment.

Contents

1 Introduction **3**

2 What and Why of DevOps? **3**
2.1 Why DevOps is Important? 3

3 Tools Used **4**

4 Project Architecture **4**

5 Source Code Management (GitHub) **5**
5.1 Repository Creation 5

6 Jenkins Setup **6**
6.1 Jenkins Installation using Docker 6
6.2 Access Jenkins 6
6.3 Jenkins Credential Configuration 6

7 Build Automation using Maven **7**

8 Jenkins Pipeline Creation **7**
8.1 Jenkins Pipeline Script 7

9 Docker Containerization **8**
9.1 Running Container 9
9.2 Dockerfile 9
9.3 Build Docker Image 9

10 DockerHub Repository **9**

11 Ansible Deployment **9**
11.1 Inventory File 10
11.2 Ansible Playbook 10

12 Scientific Calculator Output **11**

13 Conclusion **11**

1 Introduction

This project focuses on demonstrating the practical implementation of DevOps practices by developing and deploying a scientific calculator application through an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline. The main objective of this project is to automate the complete software delivery lifecycle, including building the application, testing it, containerizing the application, storing the container image in a registry, and finally deploying the application using automation tools.

Traditionally, software development and operations teams worked separately, which often resulted in delays, configuration inconsistencies, and deployment failures. DevOps addresses these challenges by integrating development and operations practices, enabling faster and more reliable delivery of applications. In this project, DevOps principles such as automation, continuous integration, and continuous deployment are implemented to streamline the development workflow and reduce manual intervention.

The application used in this project is a Java-based scientific calculator that performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The application is built using the Maven build tool, which compiles the source code and packages it into a runnable JAR file. Once the build process is completed, the application is containerized using Docker. Containerization allows the application to run in a consistent environment regardless of the host system, thereby eliminating issues related to dependency conflicts and environment differences.

Overall, this project provides a practical understanding of how modern DevOps tools and practices can be used together to automate application development, integration, and deployment in a real-world environment.

2 What and Why of DevOps?

DevOps is a combination of two words: Development and Operations. It is a modern software engineering approach that focuses on improving collaboration between software developers (who build applications) and IT operations teams (who deploy and maintain them).

Traditionally, development and operations teams worked separately. Developers were responsible for writing the code, while operations teams handled deployment and maintenance. This separation often caused delays, communication gaps, and deployment failures. DevOps was introduced to solve these problems by bringing both teams together and encouraging collaboration throughout the entire software development lifecycle.

DevOps is a set of practices, tools, and cultural philosophies that aims to automate and streamline the process of building, testing, deploying, and monitoring applications. Its main goal is to shorten the development lifecycle while continuously delivering high-quality software.

2.1 Why DevOps is Important?

- Faster software delivery
- Continuous Integration and Continuous Deployment
- Automation of build, test and deployment

- Improved collaboration between development and operations teams
- Reliable and scalable systems

3 Tools Used

- GitHub – Source Code Management
- Jenkins – Continuous Integration Pipeline
- Docker – Containerization
- DockerHub – Container Registry
- Ansible – Configuration Management and Deployment
- Maven – Build Tool

4 Project Architecture

```
Developer
   |
   v
GitHub (Source Code Repository)
   |
   v
Jenkins Pipeline (CI/CD)
   |
   |--- Build & Test (Maven)
   |
   v
Docker Build
   |
   v
DockerHub (Image Repository)
   |
   v
Ansible Deployment
   |
   v
Docker Container
   |
   v
Scientific Calculator Application Running
```

Pipeline flow:

- Developer pushes code to GitHub
- Jenkins pipeline triggers
- Maven builds the project
- Docker builds the container image
- DockerHub stores the image
- Ansible deploys the container

5 Source Code Management (GitHub)

GitHub is used to store and manage the project source code.

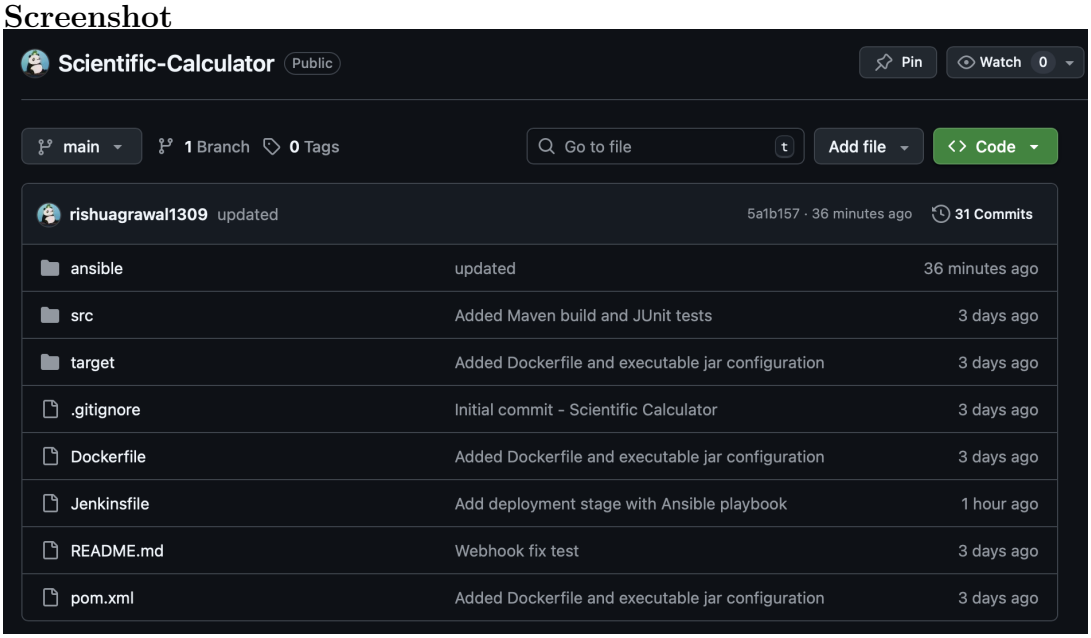
5.1 Repository Creation

- Create a new repository
- Push the project files

Commands Used

```
git init
git add .
git commit -m "Initial commit"
git push origin main
```

Screenshot



Repository Link:
`https://github.com/rishuagrawal1309/Scientific-Calculator`

6 Jenkins Setup

Jenkins is an open-source automation server used to implement Continuous Integration and Continuous Deployment pipelines. In this project, Jenkins automatically pulls the latest code from GitHub, builds the project using Maven, creates a Docker image and deploys it using Ansible.

6.1 Jenkins Installation using Docker

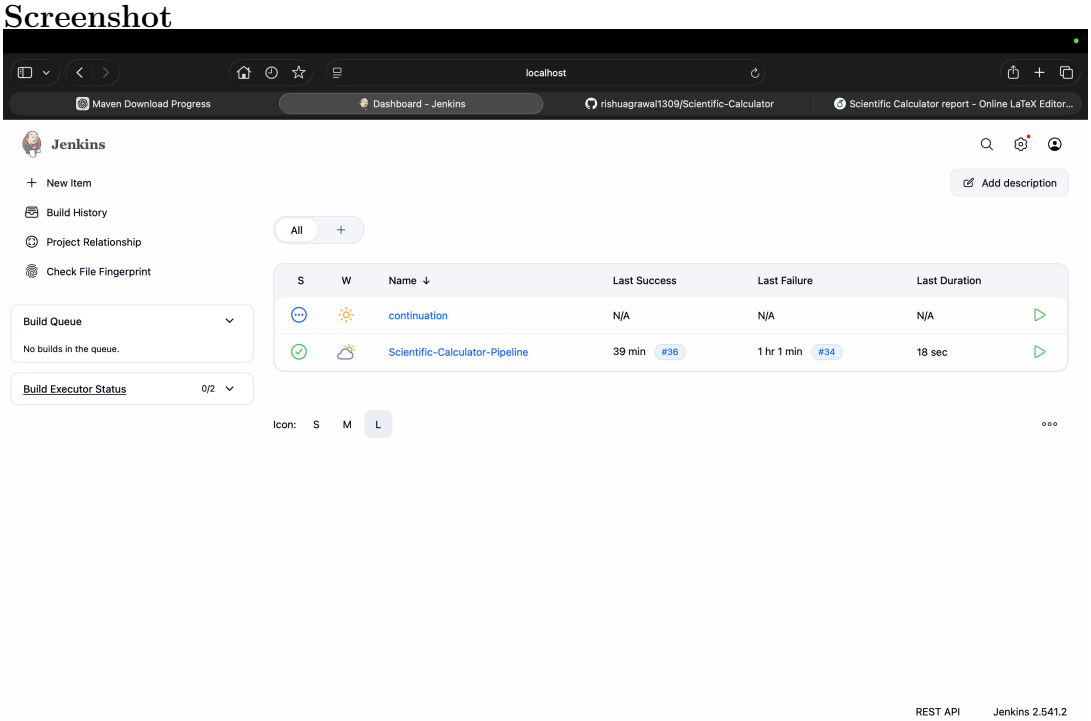
```
docker run -d \
-p 8080:8080 \
-p 50000:50000 \
--name myjenkins \
jenkins/jenkins:lts
```

6.2 Access Jenkins

Open in browser:

```
http://localhost:8080
```

Screenshot



6.3 Jenkins Credential Configuration

To allow Jenkins to push Docker images to DockerHub, DockerHub credentials are stored securely in Jenkins credentials manager.

Steps:

- Navigate to Manage Jenkins

- Click Manage Credentials
- Add Username and Password credentials
- Set ID as dockerhub-cred

7 Build Automation using Maven

Apache Maven is used as the build tool for compiling the Java application and packaging it into a JAR file.

The following command compiles the project and generates the executable JAR file.

```
mvn clean package
```

8 Jenkins Pipeline Creation

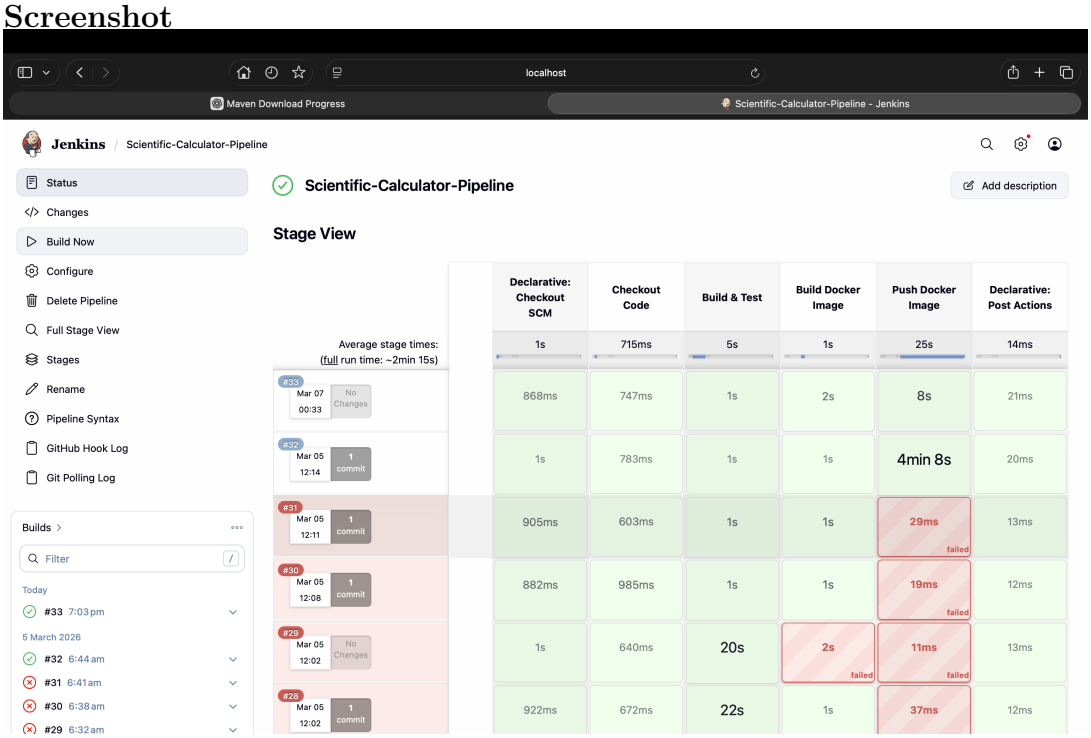
A pipeline is created to automate build, test, docker image creation and deployment.

8.1 Jenkins Pipeline Script

```
pipeline {
    agent any

    stages {

        stage('Checkout Code') {
            steps {
                checkout scm
            }
        }

        stage('Build & Test') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh '''
                docker build -t rishuagrawal1309/scientific-
                   calculator:${BUILD_NUMBER} .
                '''
            }
        }

        stage('Push Docker Image') {
            steps {
```

```
                withCredentials([usernamePassword(credentialsId:
                    'dockerhub-cred', usernameVariable: 'USER',
                    passwordVariable: 'PASS')]) {
                    sh '''
                    echo $PASS | docker login -u $USER --password
                        -stdin
                    docker push rishuagrawal1309/scientific-
                        calculator:${BUILD_NUMBER}
                    '''
                }
            }
        }

        stage('Deploy with Ansible') {
            steps {
                sh '''
                ansible-playbook ansible/deploy_calculator.yml \
                -i ansible/inventory \
                -e build_number=${BUILD_NUMBER}
                '''
            }
        }
    }
}
```

Screenshot



9 Docker Containerization

Docker is used to package the application into a container.

9.1 Running Container

The Docker container is started using the following command.

```
docker run -d --name scientific_calculator scientific-calculator
```

9.2 Dockerfile

```
FROM eclipse-temurin:17-jdk

WORKDIR /app

COPY target/scientific-calculator-1.0.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

9.3 Build Docker Image

```
docker build -t scientific-calculator .
```

10 DockerHub Repository

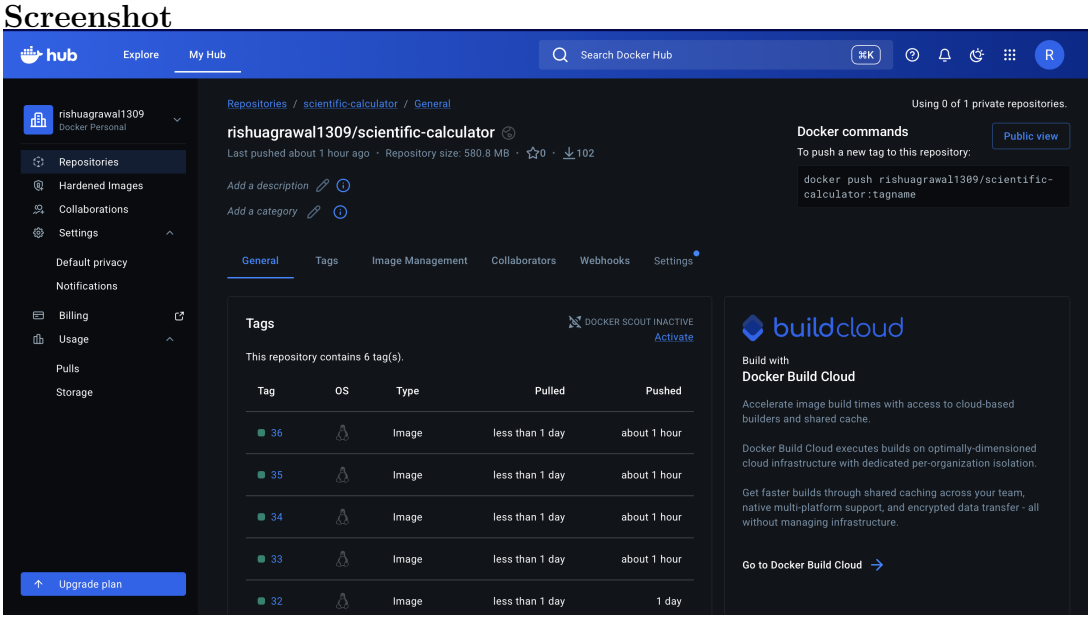
DockerHub stores the built Docker images.

Repository Link:

`https://hub.docker.com/r/rishuagrawal1309/scientific-calculator`

```
docker push rishuagrawal1309/scientific-calculator:34
```

Screenshot



11 Ansible Deployment

Ansible is used to deploy the Docker container automatically.

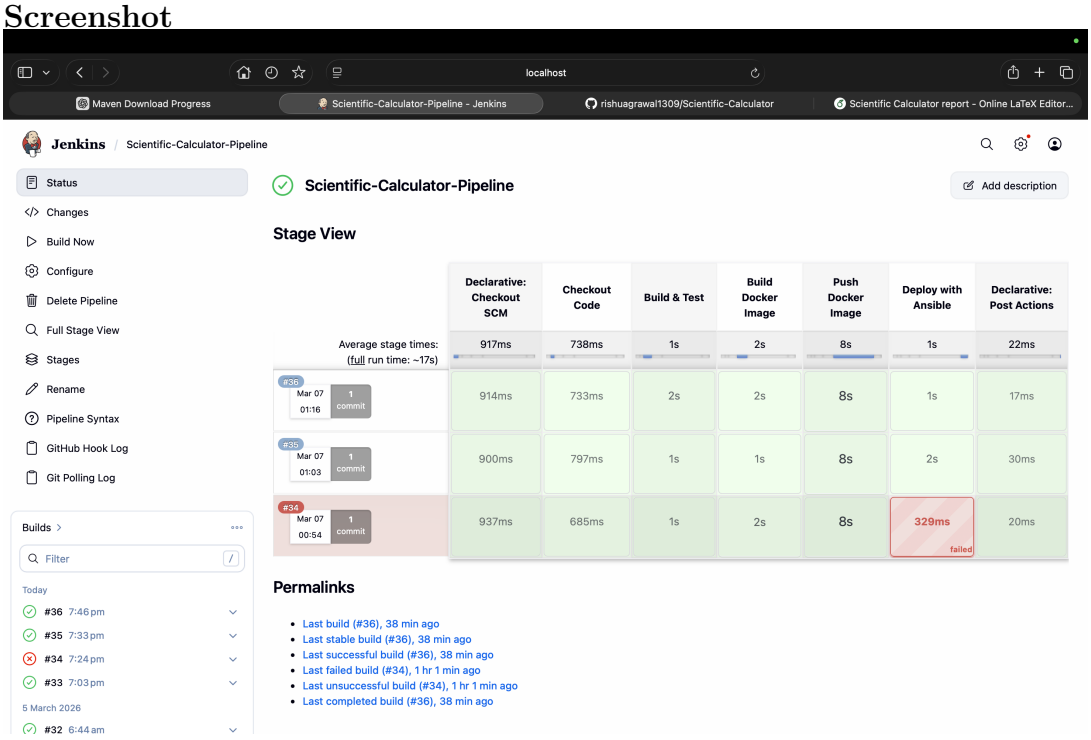
11.1 Inventory File

```
localhost ansible_connection=local
```

11.2 Ansible Playbook

```
---
- name: Deploy Scientific Calculator Container
  hosts: localhost
  become: false

  vars:
    image_tag: "{{ build_number }}"

  tasks:

    - name: Remove old container if exists
      community.docker.docker_container:
        name: scientific_calculator
        state: absent

    - name: Run Scientific Calculator Container
      community.docker.docker_container:
        name: scientific_calculator
        image: "rishuagrawal1309/scientific-calculator:{{
           image_tag }}"
        state: started
        interactive: true
        tty: true
```

Screenshot



12 Scientific Calculator Output

Below is the output of the scientific calculator application running inside Docker container.

Commands

```
docker exec -it scientific_calculator java -jar app.jar
```

Screenshot

```
rishuagrawal13@Rishus-MacBook-Air Scientific-Calculator % docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED          STATUS          PORTS
                                                   NAMES
ac53f8d3391a   rishuagrawal1309/scientific-calculator:36   "java -jar app.jar"      27 seconds ago   Up 27 seconds
                                                   scientific_calculator
19c9b8b3db61   jenkins/jenkins:lts                        "/usr/bin/tini -- /u…"   38 hours ago     Up 45 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp,
 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp   myjenkins
rishuagrawal13@Rishus-MacBook-Air Scientific-Calculator % docker exec -it scientific_calculator bash
root@ac53f8d3391a:/app# ls
app.jar
root@ac53f8d3391a:/app# java -jar app.jar

--- Scientific Calculator ---
1. Square Root (√x)
2. Factorial (!x)
3. Natural Log (ln x)
4. Power (x^b)
5. Exit
Choose option: 3
Enter number: 10
Result: 2.302585092994046

--- Scientific Calculator ---
1. Square Root (√x)
2. Factorial (!x)
3. Natural Log (ln x)
4. Power (x^b)
5. Exit
Choose option: 5
Exiting...
root@ac53f8d3391a:/app# exit
exit
rishuagrawal13@Rishus-MacBook-Air Scientific-Calculator % docker stop scientific_calculator
scientific_calculator
rishuagrawal13@Rishus-MacBook-Air Scientific-Calculator %
```

13 Conclusion

This project successfully demonstrates the design and implementation of a complete DevOps pipeline for building, packaging, and deploying a scientific calculator application. The project integrates several modern DevOps tools and technologies to automate different stages of the software development lifecycle. By combining tools such as GitHub, Jenkins, Maven, Docker, Docker Hub, and Ansible, the project illustrates how development and operations activities can be streamlined to achieve faster and more reliable software delivery.

The DevOps pipeline begins with source code management using GitHub, where the scientific calculator application's source code is stored and version controlled. GitHub enables developers to maintain a centralized repository, track code changes, and collaborate effectively.

During the Continuous Integration stage, Jenkins automatically pulls the latest code from the GitHub repository and builds the application using the Maven build tool. Maven compiles the Java source code, resolves dependencies, runs the necessary build steps, and generates a runnable JAR file.

After the build stage is completed successfully, Docker is used to containerize the application. Docker packages the application and its dependencies into a lightweight container image, ensuring that the application can run consistently across different computing environments.

Once the Docker image is created, it is pushed to Docker Hub, which serves as a cloud-based container registry. Docker Hub stores and manages container images, allowing them to be easily distributed and accessed from different systems.

For the deployment stage, Ansible is used as an automation and configuration management tool. Ansible playbooks define the tasks required to pull the Docker image from

Docker Hub and run the container on the target machine.

Another important advantage of this DevOps pipeline is the reduction of manual intervention throughout the development process. Each stage of the pipeline, from code integration to deployment, is automated, enabling faster delivery of updates and improvements. The pipeline also ensures that errors can be detected early during the build process, allowing developers to fix issues quickly.

Overall, this project provides practical exposure to real-world DevOps practices and highlights the importance of automation in modern software development. By implementing a CI/CD pipeline, the project demonstrates how organizations can improve development efficiency, reduce deployment time, ensure consistency across environments, and deliver high-quality software in a reliable and scalable manner. The knowledge and experience gained through this project can be applied to more complex software systems and large-scale enterprise applications in the future.